

INFORME DE LABORATORIO

isiL03 - Legacy System Update - IA Socrático – evidence

Universidad de los Llanos – UNILLANOS

Programa: Ingeniería de Sistemas

Estudiante: Hanner Contreras

Código: 160005312

Asignatura: Ingeniería de Software

Docente: OLGA LUCERO VEGA MARQUEZ

Fecha: 24 de agosto de 2026

ENTREGABLE 1: FASE 1 – AUDITORÍA DE MEMORIA Y DIÁLOGO SOCRÁTICO (C++) (PARA EL REPOSITORIO)

1.1. Código C++ auditado (Archivo: SistemaBancarioLegacy.cpp)

```
// SistemaBancarioLegacy.cpp - Código a auditar por los estudiantes
#include <iostream>
#include <fstream>
#include <string>

struct CuentaLegacy {
    char* titular;
    double saldo;
    int tipoCuenta; // 1: Ahorros, 2: Corriente
    double limiteSobregiro;
};

CuentaLegacy* crearCuenta(const char* nombre, double saldoInicial, int tipo) {
    CuentaLegacy* c = new CuentaLegacy();
    c->titular = new char[50];
    strcpy(c->titular, nombre);
    c->saldo = saldoInicial;
    c->tipoCuenta = tipo;
    c->limiteSobregiro = (tipo == 2) ? 500.0 : 0.0;
    return c;
}
```

```

}

void procesarRetiro(CuentaLegacy* c, double monto) {
    if (c->tipoCuenta == 1 && c->saldo >= monto) {
        c->saldo -= monto;
    } else if (c->tipoCuenta == 2 && (c->saldo + c->limiteSobregiro) >= monto) {
        c->saldo -= monto; // Fuga potencial de lógica y sin registro de auditoría
    }
    // NOTA: No hay liberación de memoria de 'titular' ni de 'c' al terminar el programa
}

```

1.2. Análisis Detallado de Auditoría

Se han identificado cuatro categorías principales de problemas. Cada tabla describe el hallazgo, su impacto y la recomendación.

Tabla 1: Fugas de Memoria y Gestión de Recursos

Líneas	Fragmento de código	Tipo de problema	Descripción detallada	Impacto en el sistema	Recomendación
7	char* titular;	Puntero sin destructor asociado	El miembro titular es un puntero a char que se asignará dinámicamente. No hay un destructor que libere esta memoria.	Cada cuenta creada pierde el bloque de memoria del nombre cuando el objeto se destruye o se pierde la referencia. En un servidor con miles de cuentas, esto puede agotar la memoria.	Usar std::string en lugar de char* o implementar un destructor que libere la memoria con delete[].
13	CuentaLegacy* c = new CuentaLegacy();	Fuga de memoria del objeto principal	Se asigna memoria en el heap para el struct CuentaLegacy mediante new. No existe un delete correspondiente.	El objeto nunca se libera, provocando que toda la estructura permanezca retenida en memoria.	Crear una función liberarCuenta(CuentaLegacy*) que realice delete y llamarla al finalizar el uso del objeto.

Líneas	Fragmento de código	Tipo de problema	Descripción detallada	Impacto en el sistema	Recomendación
14	c->titular = new char[50];	Fuga de memoria del arreglo de caracteres	Se asigna un arreglo de 50 caracteres para almacenar el nombre. Nunca se realiza el delete[] correspondiente.	El bloque de memoria destinado al nombre queda perdido, independientemente de la vida útil del objeto principal.	Liberar la memoria mediante delete[] c->titular en el destructor o en una función de limpieza.
24-28	Función procesarRetiro	Ausencia de liberación en el flujo de retiro	La función modifica el saldo, pero no gestiona la liberación de la cuenta ni de los recursos asociados a ella.	Si la cuenta fue creada dinámicamente y se utiliza únicamente para realizar un retiro, sus recursos pueden quedar sin liberar al finalizar su uso.	Implementar un mecanismo de gestión del ciclo de vida de los objetos basado en RAII , evitando la gestión manual de memoria cuando sea posible.

Tabla 2: Vulnerabilidades de Seguridad y Corrupción de Memoria

Líneas	Fragmento de código	Tipo de problema	Descripción detallada	Impacto en el sistema	Recomendación
15	strcpy(c->titular, nombre);	Desbordamiento de búfer (buffer overflow)	strcpy no verifica el tamaño del texto de origen. Si nombre contiene más de 49 caracteres, se escribirá fuera del arreglo titular[50], pudiendo corromper zonas de memoria adyacentes.	Puede sobrescribir otros campos del struct o áreas críticas de memoria, provocando fallos del programa y posibles vulnerabilidades de seguridad.	Usar una alternativa que limite la cantidad de caracteres, como strncpy, o preferiblemente utilizar std::string para gestionar automáticamente la memoria.
12-13	crearCuenta sin validación de nombre	Falta de verificación de puntero nulo	No se comprueba si nombre es nullptr. Si se recibe un puntero nulo,	El programa puede finalizar inesperadamente (<i>segmentation fault</i>), afectando la	Validar que nombre no sea nullptr antes de utilizarlo, por ejemplo mediante

Líneas	Fragmento de código	Tipo de problema	Descripción detallada	Impacto en el sistema	Recomendación
			posteriormente strcpy intentará acceder a una dirección de memoria inválida, provocando un error de ejecución.	disponibilidad y estabilidad del sistema bancario.	una condición de validación.

Tabla 3: Problemas de Arquitectura y Diseño (POO vs. Procedimental)

Líneas	Fragmento de código	Tipo de problema	Descripción detallada	Impacto en el sistema	Recomendación
5–10	struct CuentaLegacy { ... };	Violación de encapsulamiento	Todos los miembros son públicos por defecto al utilizar struct. Cualquier función externa puede modificar directamente atributos como saldo o tipoCuenta sin ningún mecanismo de control.	Se pierde la integridad de los datos. Un error en una función externa podría dejar la cuenta en un estado inconsistente, por ejemplo, permitiendo un saldo negativo en una cuenta de ahorros.	Utilizar una class con atributos private y métodos públicos que controlen el acceso y apliquen las reglas de negocio.
12–20	crearCuenta	Función global en lugar de constructor	La creación de las cuentas se realiza mediante una función independiente, en lugar de utilizar un constructor que garantice la inicialización correcta de todos los atributos del objeto.	La inicialización queda dispersa. Si posteriormente se agregan nuevos atributos, será necesario modificar los diferentes puntos donde se crean las cuentas.	Definir una clase con un constructor que reciba los parámetros necesarios y garantice una inicialización válida del objeto.
22–28	procesarRetiro	Lógica de negocio acoplada a un switch con enteros	El tipo de cuenta se determina mediante valores	Se viola el Principio Abierto/Cerrado	Utilizar herencia y polimorfismo, permitiendo que

Líneas	Fragmento de código	Tipo de problema	Descripción detallada	Impacto en el sistema	Recomendación
			enteros, por ejemplo 1 para Ahorros y 2 para Corriente. Para agregar un nuevo tipo de cuenta, sería necesario modificar esta función incorporando nuevas condiciones.	(OCP). El sistema se vuelve rígido, aumenta el acoplamiento y resulta más difícil de extender y mantener.	cada tipo de cuenta implemente su propio comportamiento para el método retirar.
—	Ausencia de clases	Diseño totalmente procedimental	No existe una abstracción clara de los conceptos bancarios. La lógica de negocio, la gestión manual de memoria y otras responsabilidades se encuentran mezcladas.	El mantenimiento se vuelve más costoso y propenso a errores. Además, el código presenta poca reutilización y escalabilidad.	Migrar progresivamente hacia un diseño orientado a objetos utilizando clases, encapsulamiento, herencia y polimorfismo en Java o C++.

Tabla 4: Deficiencias en Lógica de Negocio y Auditoría

Líneas	Fragmento de código	Tipo de problema	Descripción detallada	Impacto en el sistema	Recomendación
24–26	if (c->tipoCuenta == 1 && c->saldo >= monto)	Comportamiento silencioso ante fallo de retiro	Si una cuenta de ahorros no dispone de saldo suficiente, la operación simplemente se ignora. No se lanza una excepción ni se informa al usuario que el retiro no pudo realizarse.	El cliente no sabe que la operación fue rechazada. Esto puede generar confusión, transacciones no registradas correctamente y posibles conflictos en la gestión de las operaciones.	Lanzar una excepción o devolver un resultado/código de error que permita informar claramente el motivo del rechazo.

Líneas	Fragmento de código	Tipo de problema	Descripción detallada	Impacto en el sistema	Recomendación
27	c->saldo -= monto; (en cuenta corriente)	Falta de aplicación de interés de mora	Cuando la cuenta corriente entra en sobregiro, el código permite modificar el saldo sin aplicar el interés de mora diario establecido por la regla de negocio.	El banco puede dejar de registrar ingresos correspondientes a los intereses y el saldo de la cuenta no representa correctamente las obligaciones del cliente.	Implementar la lógica correspondiente al cálculo y cobro del interés de mora cuando se produzca un sobregiro.
24–28	Todo el bloque	Ausencia de registro de auditoría	No se registra información sobre las operaciones realizadas, como quién efectuó el retiro, el monto, la fecha, la cuenta involucrada o el saldo resultante.	Se pierde la trazabilidad de las operaciones. Esto dificulta las auditorías, la investigación de incidentes y el cumplimiento de requisitos de control financiero.	Implementar un sistema de registro de operaciones (logging) que almacene la información relevante de cada transacción de manera segura.
22	void procesarRetiro(CuentaLegacy* c, double monto)	Falta de validación de puntero nulo	La función no verifica que c sea diferente de nullptr antes de acceder a sus atributos. Si recibe un puntero nulo, intentará acceder a una dirección de memoria inválida.	Puede producir un error de ejecución y provocar la interrupción del programa o del servicio que procesa las operaciones.	Validar que c no sea nullptr antes de utilizarlo o utilizar un mecanismo de manejo de excepciones apropiado.

Conclusión de la Auditoría:

El código C++ legacy presenta graves deficiencias que lo hacen inapropiado para un entorno de producción bancaria: fugas de memoria, vulnerabilidades de seguridad, arquitectura procedural rígida y ausencia de auditoría. Estos hallazgos justifican plenamente la migración a un diseño orientado a objetos en Java.

1.3. Diálogo Socrático con IA

Se configuró una Gema en Gemini con el siguiente prompt:

"Actúa como un auditor de software socrático. No des soluciones de código. Solo haz preguntas para guiarme a encontrar los fallos en este C++ legacy. Quiero que me cuestiones sobre la gestión de memoria, el encapsulamiento y la mantenibilidad."

Las preguntas realizadas a la IA fueron:

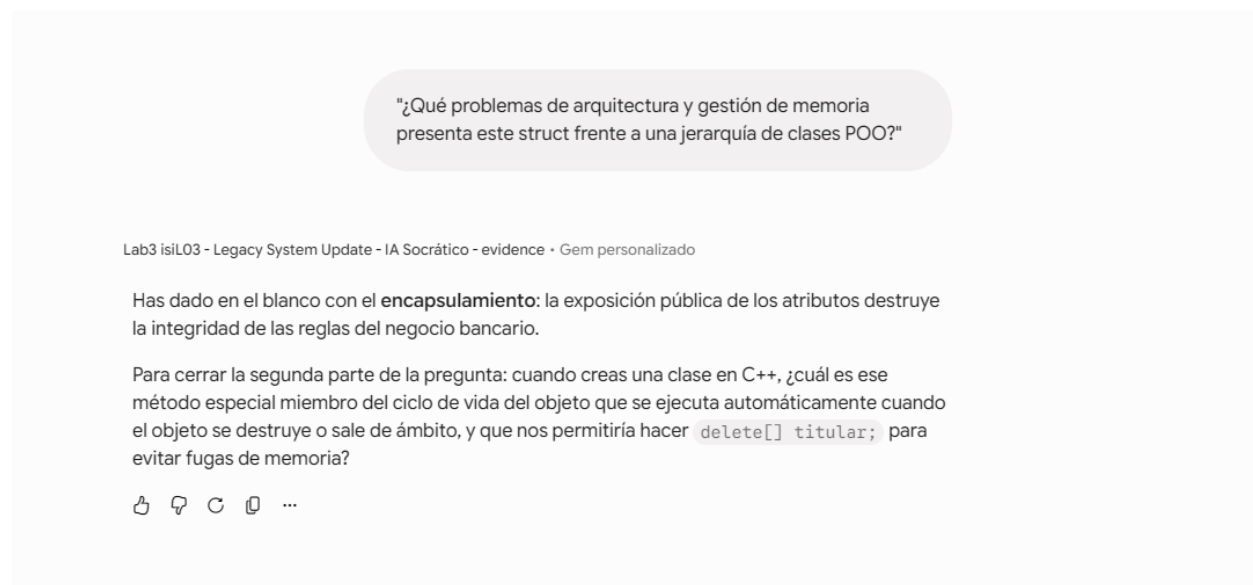


Figura 1. Pregunta socrática de arquitectura y gestión

2. FASE 2: REDISEÑO E IMPLEMENTACIÓN EN JAVA

Se ha migrado el sistema a una arquitectura orientada a objetos en Java 21, aplicando herencia, clases abstractas y polimorfismo. Todos los atributos son privados y se accede mediante getters/setters.

2.1. Jerarquía de Clases

```
package bancario;

public abstract class CuentaBancaria {
    private String numeroCuenta;
    private String titular;
    private double saldo;

    public CuentaBancaria(String numeroCuenta, String titular, double saldoInicial) {
        this.numeroCuenta = numeroCuenta;
        this.titular = titular;
        this.saldo = saldoInicial;
    }

    public String getNumeroCuenta() { return numeroCuenta; }
    public String getTitular() { return titular; }
    public double getSaldo() { return saldo; }
    protected void setSaldo(double saldo) { this.saldo = saldo; }

    public void depositar(double monto) {
        if (monto <= 0) throw new IllegalArgumentException("Monto a depositar debe ser positivo");
        this.saldo += monto;
    }

    public abstract void aplicarComisionMensual();
    public abstract void retirar(double monto) throws SaldoInsuficienteException;
}
```

Figura 2. Clase abstracta CuentaBancaria

Excepción personalizada

```
package bancario;

public class SaldoInsuficienteException extends Exception {
    public SaldoInsuficienteException(String mensaje) {
        super(mensaje);
    }
}
```

Figura 3. Excepción para Saldo Insuficiente

2.2. Subclases Especializadas

CuentaAhorros (No permite sobregiro; aplica interés y comisión fija)

```
package bancario;

public class CuentaAhorros extends CuentaBancaria {
    private double tasaInteres;
    private double comisionMensual;

    public CuentaAhorros(String numeroCuenta, String titular, double saldoInicial,
        double tasaInteres, double comisionMensual) {
        super(numeroCuenta, titular, saldoInicial);
        this.tasaInteres = tasaInteres;
        this.comisionMensual = comisionMensual;
    }

    @Override
    public void aplicarComisionMensual() {
        double nuevoSaldo = (getSaldo() * (1 + tasaInteres)) - comisionMensual;
        setSaldo(Math.max(nuevoSaldo, 0)); // No permite sobregiro
    }

    @Override
    public void retirar(double monto) throws SaldoInsuficienteException {
        if (monto <= 0) throw new IllegalArgumentException("Monto debe ser positivo");
        if (getSaldo() < monto) {
            throw new SaldoInsuficienteException("Saldo insuficiente en Cuenta de Ahorros.");
        }
        setSaldo(getSaldo() - monto);
    }
}
```

Figura 4. Clase CuentaAhorros

CuentaCorriente (Permite sobregiro con cupo; cobra interés de mora diario)

```
package bancario;

public class CuentaCorriente extends CuentaBancaria {
    private double cupoSobregiro;
    private double comisionSobregiro; // Tasa diaria

    public CuentaCorriente(String numeroCuenta, String titular, double saldoInicial,
                           double cupoSobregiro, double comisionSobregiro) {
        super(numeroCuenta, titular, saldoInicial);
        this.cupoSobregiro = cupoSobregiro;
        this.comisionSobregiro = comisionSobregiro;
    }

    @Override
    public void retirar(double monto) throws SaldoInsuficienteException {
        if (monto <= 0) throw new IllegalArgumentException("Monto debe ser positivo");
        double nuevoSaldo = getSaldo() - monto;

        // Si ya estaba en sobregiro, cobrar interés de mora por este retiro
        if (getSaldo() < 0) {
            double interesMora = Math.abs(getSaldo()) * comisionSobregiro;
            nuevoSaldo -= interesMora;
        }

        if (nuevoSaldo < -cupoSobregiro) {
            throw new SaldoInsuficienteException("Cupo de sobregiro excedido.");
        }
        setSaldo(nuevoSaldo);
    }

    @Override
    public void aplicarComisionMensual() {
        if (getSaldo() < 0) {
            double interesAcumulado = Math.abs(getSaldo()) * comisionSobregiro * 30;
            double nuevoSaldo = getSaldo() - interesAcumulado;
            setSaldo(Math.max(nuevoSaldo, -cupoSobregiro));
        }
    }
}
```

Figura 5. Clase Corriente

3. FASE 3: GESTIÓN DE RECURSOS CON AUTOCLOSEABLE

Se implementó la interfaz `AutoCloseable` para garantizar el cierre seguro del archivo de auditoría, equivalente al `RAII` de C++.

3.1. Clase `RegistroAuditoriaBancaria`

```
package bancario;

import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.time.LocalDateTime;

public class RegistroAuditoriaBancaria implements AutoCloseable {
    private PrintWriter writer;

    public RegistroAuditoriaBancaria(String archivo) throws IOException {
        this.writer = new PrintWriter(new FileWriter(archivo, true));
    }

    public void registrar(String operacion, String cuenta, double monto, double saldoFinal) {
        writer.printf("[%s] %s | Cuenta: %s | Monto: %.2f | Saldo: %.2f%n",
            LocalDateTime.now(), operacion, cuenta, monto, saldoFinal);
        writer.flush();
    }

    public void registrarMensaje(String mensaje) {
        writer.printf("[%s] %s%n", LocalDateTime.now(), mensaje);
        writer.flush();
    }

    @Override
    public void close() {
        if (writer != null) {
            writer.close();
            System.out.println(">>> Recurso de auditoría cerrado (RAII simulado en Java).");
        }
    }
}
```

Figura 6. Clase `RegistroAuditoriaBancaria`

3.2. Uso con try-with-resources (Main)

```
package bancario;

public class Main {
    public static void main(String[] args) {
        CuentaAhorros ahorros = new CuentaAhorros("AH-001", "Carlos Pérez", 1000.0, 0.02, 5.0);
        CuentaCorriente corriente = new CuentaCorriente("CC-001", "María Gómez", 500.0, 1000.0, 0.00
1);

        try (RegistroAuditoriaBancaria log = new RegistroAuditoriaBancaria("auditoria.log")) {

            log.registrarMensaje("=== INICIO JORNADA ===");

            ahorros.depositar(200);
            log.registrar("DEPOSITO", ahorros.getNumeroCuenta(), 200, ahorros.getSaldo());

            try { ahorros.retirar(1500); }
            catch (SaldoInsuficienteException e) {
                log.registrarMensaje("ERROR: " + e.getMessage());
            }

            ahorros.aplicarComisionMensual();
            log.registrar("COMISION", ahorros.getNumeroCuenta(), 0, ahorros.getSaldo());

            corriente.depositar(300);
            log.registrar("DEPOSITO", corriente.getNumeroCuenta(), 300, corriente.getSaldo());

            try { corriente.retirar(1200); }
            catch (SaldoInsuficienteException e) {
                log.registrarMensaje("ERROR: " + e.getMessage());
            }

            corriente.aplicarComisionMensual();
            log.registrar("COMISION", corriente.getNumeroCuenta(), 0, corriente.getSaldo());

            log.registrarMensaje("=== FIN JORNADA ===");

        } catch (Exception e) {
            e.printStackTrace();
        }

        System.out.println("Saldo Ahorros: " + ahorros.getSaldo());
        System.out.println("Saldo Corriente: " + corriente.getSaldo());
    }
}
```

Figura 7. Clase Main

4. CONCLUSIONES

- La auditoría del código C++ legacy reveló graves fugas de memoria, vulnerabilidades de seguridad y un diseño procedural que dificulta la extensión y el mantenimiento.
- La migración a Java con una jerarquía de clases abstractas y subclasses especializadas permite aplicar correctamente las reglas de negocio (ahorros sin sobregiro y corriente con cupo e interés de mora), mejorando la encapsulación y el polimorfismo.
- El uso de AutoCloseable con try-with-resources garantiza el cierre automático del archivo de auditoría, simulando el comportamiento RAII de C++ y evitando dependencias del Garbage Collector.
- La interacción socrática con la IA resultó valiosa para identificar problemas de diseño sin obtener soluciones directas, fomentando el pensamiento crítico y el aprendizaje autónomo.
- El sistema refactorizado en Java es más seguro, mantenible y extensible, cumpliendo con los estándares de la industria para sistemas bancarios modernos.

5. Entregables del laboratorio

1. Repositorio de código: Código fuente C++ auditado y código Java refactorizado en un repositorio Git.

- Adjuntado en el repositorio:
https://github.com/hscarvajal-sudo/Lab3_Bancario_HannerContreras.git

2. Informe técnico: Documento técnico justificando el rediseño y los hallazgos de memoria.

- Actualmente este es el informe técnico.

3. Enlaces de IA socrática: Enlace a la Gema compartida con el docente y enlace público al historial del chat interactuado.

- Adjuntado en el siguiente enlace:
<https://gemini.google.com/share/d/1VGq8ltpPiM7JIEtnRd9ZubXzIINN4Q?usp=sharing>

6. Rúbrica de evaluación

Criterio de evaluación	Descripción del nivel esperado	Ponderación
Auditoría C++ y memoria	Identificación precisa de memory leaks y justificación técnica en el informe.	20%
Arquitectura POO Java	Correcta aplicación de clases abstractas, herencia, encapsulamiento y polimorfismo.	40%
Gestión de recursos	Implementación limpia de AutoCloseable con try-with-resources.	20%
Uso socrático de IA	Interacción de calidad (Maratonista mental) verificable mediante el enlace público.	20%